

**LAPORAN PRAKTIKUM SISTEM OPERASI
DINING PHILOSOPHERS PROBLEM**



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Yusa Salasa (2410651018)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

PENDAHULUAN	
1.1 Latar Belakang	
1.2 Tujuan Praktikum	
1.3 Dasar Teori	
PERMASALAHAN DINING PHILOSOPHERS	
2.1 Deskripsi Masalah	
2.2 Kondisi Terjadinya Deadlock	
2.3 Analisis Empat Syarat Deadlock	
IMPLEMENTASI PROGRAM	
3.2 Kode Program	
3.2.1 Versi Deadlock (Demonstrasi)	
3.2.2 Versi Resource Ordering (Pencegahan Deadlock)	
3.2.3 Versi Limiting Philosophers (Pencegahan dengan Semaphore)	
ANALISIS DAN PEMBAHASAN	
4.1 Tabel Perbandingan Efisiensi dan Fairness	
4.2 Penyebab Deadlock	
4.3 Cara Kerja Setiap Solusi	
4.4 Solusi yang Paling Adil	
4.5 Kemungkinan Terjadinya Starvation	
KESIMPULAN DAN SARAN	
5.1 Kesimpulan	
5.2 Saran	
DAFTAR PUSTAKA	

PENDAHULUAN

1.1 Latar Belakang

Dalam sistem operasi modern, **sinkronisasi** merupakan aspek penting untuk mengatur proses atau *thread* yang berjalan secara bersamaan agar tidak saling mengganggu. Salah satu studi kasus klasik yang sering digunakan untuk menjelaskan konsep ini adalah **Dining Philosophers Problem**.

Permasalahan ini menggambarkan kondisi di mana beberapa proses bersaing untuk memperoleh **sumber daya bersama** (*shared resources*), seperti sumpit atau *forks*, sehingga berpotensi menimbulkan **deadlock**.

Melalui studi ini, mahasiswa dapat mempelajari bagaimana mekanisme seperti **mutual exclusion**, **semaphore**, dan **resource ordering** diterapkan untuk mencegah konflik dan kebuntuan dalam sistem paralel.

1.2 Tujuan Praktikum

Tujuan dilaksanakannya praktikum ini adalah sebagai berikut:

1. Memahami prinsip dasar sinkronisasi antar proses atau *thread* dalam sistem operasi.
2. Mengetahui kondisi yang dapat menyebabkan terjadinya **deadlock** pada sistem paralel.
3. Mengimplementasikan algoritma **pencegahan deadlock** melalui studi kasus *Dining Philosophers Problem*.
4. Membandingkan hasil eksekusi antara kondisi **deadlock** dan **deadlock prevention** untuk memahami efektivitas metode sinkronisasi.

1.3 Dasar Teori

- **Deadlock** adalah keadaan ketika dua atau lebih proses saling menunggu sumber daya yang sedang digunakan oleh proses lain, sehingga tidak ada satu pun proses yang dapat melanjutkan eksekusinya.
- Menurut **Coffman (1971)**, terdapat **empat kondisi** yang harus terpenuhi agar *deadlock* dapat terjadi, yaitu:
 1. **Mutual Exclusion:** Sumber daya hanya dapat digunakan oleh satu proses pada satu waktu.
 2. **Hold and Wait:** Sebuah proses memegang satu sumber daya dan menunggu sumber daya lain.
 3. **No Preemption:** Sumber daya tidak dapat direbut secara paksa dari proses yang sedang menggunakannya.
 4. **Circular Wait:** Terjadi rantai tunggu di mana setiap proses menunggu sumber daya yang dipegang oleh proses berikutnya.

- **Semaphore** dan **Mutex** digunakan sebagai mekanisme pengendali akses terhadap sumber daya bersama, agar tidak terjadi konflik antar proses.
- **Dining Philosophers Problem** sering dijadikan model pembelajaran untuk memahami sinkronisasi dan mencegah *deadlock*, antara lain melalui:
 1. **Resource Ordering:** Mengatur urutan pengambilan sumber daya.
 2. **Limiting Philosophers:** Membatasi jumlah proses yang dapat mengakses sumber daya pada saat bersamaan.

PERMASALAHAN DINING PHILOSOPHERS

2.1 Deskripsi Masalah

Dalam skenario *Dining Philosophers Problem*, terdapat lima filsuf yang duduk mengelilingi sebuah meja makan.

Di antara setiap pasangan filsuf terdapat satu sumpit. Untuk mulai makan, seorang filsuf harus mengambil **dua sumpit** — satu di sebelah kiri dan satu di sebelah kanan.

Permasalahan muncul ketika semua filsuf secara bersamaan mengambil sumpit di sisi kirinya. Karena setiap sumpit hanya dapat dipegang satu orang pada satu waktu, maka tidak ada filsuf yang dapat memperoleh sumpit kanan. Akibatnya, **semua filsuf menunggu tanpa akhir**, dan kondisi ini disebut **deadlock**.

2.2 Kondisi Terjadinya Deadlock

Deadlock dapat terjadi jika seluruh filsuf atau proses berada dalam kondisi berikut:

1. Masing-masing filsuf sudah memegang satu sumpit (misalnya sumpit kiri).
2. Semua filsuf menunggu sumpit kanan yang dipegang oleh tetangga di sebelah kanan.
3. Tidak ada satu pun filsuf yang bersedia melepaskan sumpit yang telah dipegang.

Kondisi ini menyebabkan seluruh proses **terblokir secara permanen**, tidak ada aktivitas makan yang dapat berlangsung, dan program berhenti tanpa menghasilkan keluaran.

2.3 Analisis Empat Syarat Deadlock

No	Kondisi Coffman	Keterpenuhan dalam Dining Philosophers
1	Mutual Exclusion	Setiap sumpit hanya dapat digunakan oleh satu filsuf dalam satu waktu.
2	Hold and Wait	Setiap filsuf memegang satu sumpit sambil menunggu sumpit lainnya.
3	No Preemption	Sumpit yang telah dipegang tidak dapat direbut oleh filsuf lain.
4	Circular Wait	Setiap filsuf menunggu sumpit yang dipegang oleh tetangga, membentuk rantai sirkular.

IMPLEMENTASI PROGRAM

3.2 Kode Program

3.2.1 Versi Deadlock (Demonstrasi)

Pada versi ini, setiap sumpit diatur menggunakan mutex, dan tiap filsuf selalu mengambil sumpit kiri terlebih dahulu, kemudian sumpit kanan.

Jika semua filsuf melakukan hal yang sama secara bersamaan, maka masing-masing akan menunggu sumpit kanan yang sedang digunakan oleh tetangganya.

Akibatnya, tidak ada filsuf yang bisa melanjutkan makan — terjadi deadlock karena semua proses saling menunggu sumber daya.

Input:

```
GNU nano 8.4
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/time.h>
#include <stdarg.h> // untuk va_start dan va_end
#include <stdint.h> // untuk intptr_t

#define N 5

pthread_mutex_t forks[];
pthread_t philo[];
int id_arr[];

struct timeval start_time;

// Fungsi untuk print dengan timestamp
void print_ts(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    struct timeval now;
    gettimeofday(&now, NULL);
    long elapsed = now.tv_sec - start_time.tv_sec;
    int mm = (elapsed / 60) % 60;
    int ss = elapsed % 60;
    printf("[%02d:%02d]", mm, ss);
    vprintf(fmt, ap);
    printf("\n");
    fflush(stdout);
    va_end(ap);
}

// Fungsi utama tiap filsuf
void* philosopher(void* arg) {
    int id = *(int*)arg; // lebih aman daripada (int)arg

    while (1) {
        print_ts("Filsuf-%d sedang berpikir...", id + 1);
        sleep(1 + (id % 2)); // pikir sebentar

        print_ts("Filsuf-%d lapar, mencoba ambil sumpit...", id + 1);

        // DEADLOCK pattern: ambil kiri dulu, lalu kanan
        int left = id;
        int right = (id + 1) % N;

        print_ts("Filsuf-%d mencoba ambil sumpit kiri (%d)", id + 1, left);
        pthread_mutex_lock(&forks[left]);
        print_ts("Filsuf-%d mengambil sumpit kiri (%d)", id + 1, left);

        // beri jeda supaya interleaving memicu deadlock
        sleep(1);

        print_ts("Filsuf-%d mencoba ambil sumpit kanan (%d)", id + 1, right);
        pthread_mutex_lock(&forks[right]);
        print_ts("Filsuf-%d mengambil sumpit kanan (%d)", id + 1, right);

        // makan
        print_ts("Filsuf-%d sedang MAKAN...", id + 1);
        sleep(2);

        // taruh sumpit
        pthread_mutex_unlock(&forks[right]);
        pthread_mutex_unlock(&forks[left]);
        print_ts("Filsuf-%d menaruh kedua sumpit", id + 1);

        // loop ulang
    }
    return NULL;
}

int main() {
    gettimeofday(&start_time, NULL);
    printf("=== DINING PHILOSOPHERS PROBLEM ===\n");
    printf("Solusi: Deadlock Demonstration\n");

    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        id_arr[i] = i;
    }

    for (int i = 0; i < N; i++) {
        pthread_create(&philo[i], NULL, philosopher, &id_arr[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_create(&philo[i], NULL, philosopher, &id_arr[i]);
        usleep(100000); // delay sedikit biar output tidak campur
    }

    // Jika deadlock terjadi, program akan HANG di pthread_join
    for (int i = 0; i < N; i++) {
        pthread_join(philo[i], NULL);
    }

    return 0;
}
```

Output:

```
yo@yoas-QEMU-Virtual-Machine:~$ nano dining_deadlock.c
yo@yoas-QEMU-Virtual-Machine:~$ ./dining_deadlock
=== DINING PHILOSOPHERS PROBLEM ===
Solusi: Deadlock Demonstration

[00:00] 🍴 Filsuf-1 sedang berpikir...
[00:00] 🍴 Filsuf-2 sedang berpikir...
[00:00] 🍴 Filsuf-3 sedang berpikir...
[00:00] 🍴 Filsuf-4 sedang berpikir...
[00:00] 🍴 Filsuf-5 sedang berpikir...
[00:01] 🍴 Filsuf-1 lapar, mencoba ambil sumpit...
[00:01] 🍴 Filsuf-1 mencoba ambil sumpit Kiri (0)
[00:01] 🍴 Filsuf-1 mengambil sumpit kiri (0)
[00:01] 🍴 Filsuf-3 lapar, mencoba ambil sumpit...
[00:01] 🍴 Filsuf-3 mencoba ambil sumpit kiri (2)
[00:01] 🍴 Filsuf-3 mengambil sumpit kiri (2)
[00:01] 🍴 Filsuf-5 lapar, mencoba ambil sumpit...
[00:01] 🍴 Filsuf-5 mencoba ambil sumpit kiri (4)
[00:01] 🍴 Filsuf-5 mengambil sumpit kiri (4)
[00:02] 🍴 Filsuf-1 mencoba ambil sumpit kanan (1)
[00:02] 🍴 Filsuf-1 mengambil sumpit kanan (1)
[00:02] 🍴 Filsuf-1 sedang MAKAN...
[00:02] 🍴 Filsuf-2 lapar, mencoba ambil sumpit...
[00:02] 🍴 Filsuf-2 mencoba ambil sumpit kiri (1)
[00:02] 🍴 Filsuf-3 mencoba ambil sumpit kanan (3)
[00:02] 🍴 Filsuf-3 mengambil sumpit kanan (3)
[00:02] 🍴 Filsuf-3 sedang MAKAN...
[00:02] 🍴 Filsuf-4 lapar, mencoba ambil sumpit...
[00:02] 🍴 Filsuf-4 mencoba ambil sumpit kiri (3)
[00:02] 🍴 Filsuf-5 mencoba ambil sumpit kanan (0)
```

Cara Compile dan run :

```
gcc -o dining_deadlock dining_deadlock.c
./dining_deadlock
```

3.2.2 Versi Resource Ordering

Pada versi ini, pola pengambilan sumpit diatur berdasarkan nomor filsuf.

Filsuf bernomor genap akan mengambil sumpit kiri lebih dulu, sedangkan filsuf bernomor ganjil mengambil sumpit kanan terlebih dahulu.

Dengan perbedaan urutan ini, rantai tunggu (circular wait) dapat dihindari sehingga deadlock tidak terjadi.

Input:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/time.h>
#include <stdlib.h>
#include <stdlib.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philo[N];
int id_arr[N];
int eat_count[ ];
struct timeval start_time;

void print_ts(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    struct timeval now;
    gettimeofday(&now, NULL);
    long elapsed = now.tv_sec - start_time.tv_sec;
    int mm = (elapsed / 60) % 60;
    int ss = elapsed % 60;
    printf("[%02d:%02d] ", mm, ss);
    vprintf(fmt, ap);
    printf("\n");
    fflush(stdout);
    va_end(ap);
}

void* philosopher(void* arg) {
    int id = (int)arg;
    unsigned int seed = id + 1;
    while (1) {
        print_ts("Filsuf-%d sedang berpikir...", id+1);
        usleep((100 + (rand_r(&seed) % 900)) * 1000);

        print_ts("Filsuf-%d lapar, mencoba ambil sumpit...", id+1);

        int left = id;

        // Resource ordering: even (0-based) pick left first, odd pick right first
        if ((id % 2) == 0) {
            print_ts("Filsuf-%d mengambil sumpit kiri (%d)", id+1, left);
            pthread_mutex_lock(&forks[left]);
            print_ts("Filsuf-%d mengambil sumpit kanan (%d)", id+1, right);
            pthread_mutex_lock(&forks[right]);
        } else {
            print_ts("Filsuf-%d mengambil sumpit kanan (%d)", id+1, right);
            pthread_mutex_lock(&forks[right]);
            print_ts("Filsuf-%d mengambil sumpit kiri (%d)", id+1, left);
            pthread_mutex_lock(&forks[left]);
        }

        // makan
        print_ts("Filsuf-%d sedang MAKAN...", id+1);
        eat_count[id]++;
        usleep((300 + (rand_r(&seed) % 700)) * 1000);

        // taruh sumpit
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        print_ts("Filsuf-%d menaruh kedua sumpit", id+1);
    }
    return NULL;
}

int main() {
    gettimeofday(&start_time, NULL);
    printf("=== DINING PHILOSOPHERS PROBLEM ===\n");
    printf("Solusi: Resource Ordering\n");
    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        id_arr[i] = i;
        eat_count[i] = 0;
    }
    for (int i = 0; i < N; i++) {
        pthread_create(&philo[i], NULL, philosopher, &id_arr[i]);
        usleep(100000);
    }

    // jalankan selama 60 detik
    sleep(60);

    // hentikan threads: kita cancel then
    for (int i = 0; i < N; i++) pthread_cancel(philo[i]);
    for (int i = 0; i < N; i++) pthread_join(philo[i], NULL);

    printf("\n=== STATISTIK (60 detik) ===\n");
    int total = 0;
    for (int i = 0; i < N; i++) {
        printf("Filsuf-%d: Makan %d kali\n", i+1, eat_count[i]);
        total += eat_count[i];
    }
    if (total > 0)
        printf("✓ Tidak ada deadlock!\n");
    else
        printf("⚠ Kemungkinan deadlock atau starvation!\n");

    // cleanup
    for (int i = 0; i < N; i++) pthread_mutex_destroy(&forks[i]);
    return 0;
}
```

Output:

```
yos@yos-QEMU-Virtual-Machine:~$ nano dining_ordering.c
yos@yos-QEMU-Virtual-Machine:~$ gcc -o dining_ordering dining_ordering.c
dining_ordering.c: In function 'philosopher':
dining_ordering.c:33:14: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast]
   33 |         int id = (int)arg;
       |                  ^
yos@yos-QEMU-Virtual-Machine:~$ ./dining_ordering
=== DINING PHILOSOPHERS PROBLEM ===
Solusi: Resource Ordering

[00:00] 🍳 Filsuf-1918107953 sedang berpikir...
[00:00] 🍳 Filsuf-1918107957 sedang berpikir...
[00:00] 🍳 Filsuf-1918107961 sedang berpikir...
[00:00] 🍳 Filsuf-1918107965 sedang berpikir...
[00:00] 🍳 Filsuf-1918107969 sedang berpikir...
[00:00] 🍴 Filsuf-1918107953 lapar, mencoba ambil sumpit...
[00:00] 🍴 Filsuf-1918107953 mengambil sumpit kiri (1918107952)
Segmentation fault (core dumped)
```

Cara compile dan run :

```
gcc -o dining_ordering dining_ordering.c
./dining_ordering
```

3.2.3 Versi Limiting Philosophers

Pada versi ini digunakan **semaphore** sebagai pengendali agar hanya **empat filsuf** yang diizinkan mencoba mengambil sumpit pada waktu yang sama.

Dengan adanya batasan tersebut, selalu ada setidaknya satu sumpit yang tersisa, sehingga **minimal satu filsuf dapat makan terlebih dahulu** dan kemudian melepaskan sumpitnya.

Pendekatan ini efektif mencegah terjadinya **deadlock** karena tidak semua filsuf menunggu sumber daya secara bersamaan.

Input:

Output:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/time.h>
#include <stdlib.h>
#include <stdint.h>

#define N 5

pthread_mutex_t forks[];
pthread_t philo[];
int id_arr[];
int eat_count[];
struct timeval start_time;

void print_ts(const char *fmt, ...) {
    va_list ap;
    va_start(ap, fmt);
    struct timeval now;
    gettimeofday(&now, NULL);
    time_t elapsed = now.tv_sec - start_time.tv_sec;
    int mm = (elapsed / 60) % 60;
    int ss = elapsed % 60;
    printf("%02d:%02d ", mm, ss);
    vprintf(fmt, ap);
    printf("\n");
    fflush(stdout);
    va_end(ap);
}

void* philosopher(void* arg) {
    int id = (int) arg;
    unsigned int seed = id + 1;
    while (1) {
        print_ts("🧠 Filsuf-%d sedang berpikir...", id+1);
        usleep((100 + (rand_r(&seed) % 900)) * 1000);

        print_ts("🍴 Filsuf-%d Lapar, mencoba ambil sumpit...", id+1);

        int left = id;
        int right = (id + 1) % N;
        // Resource ordering: even (0-based) pick left first, odd pick right first
        if ((id % 2) == 0) {
            print_ts("🍴 Filsuf-%d mengambil sumpit kiri (%d)", id+1, left);
            pthread_mutex_lock(&forks[left]);
            print_ts("🍴 Filsuf-%d mengambil sumpit kanan (%d)", id+1, right);
            pthread_mutex_lock(&forks[right]);
        } else {
            print_ts("🍴 Filsuf-%d mengambil sumpit kanan (%d)", id+1, right);
            pthread_mutex_lock(&forks[right]);
            print_ts("🍴 Filsuf-%d mengambil sumpit kiri (%d)", id+1, left);
            pthread_mutex_lock(&forks[left]);
        }

        // makan
        print_ts("🍽️ Filsuf-%d sedang MAKAN...", id+1);
        eat_count[id]++;
        usleep((100 + (rand_r(&seed) % 700)) * 1000);

        // taruh sumpit
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        print_ts("🍴 Filsuf-%d menaruh kedua sumpit", id+1);
    }
    return NULL;
}

int main() {
    gettimeofday(&start_time, NULL);
    printf("=== DINING PHILOSOPHERS PROBLEM ===\n");
    printf("Solusi: Resource Ordering\n");
    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        id_arr[i] = i;
        eat_count[i] = 0;
    }
    for (int i = 0; i < N; i++) {
        pthread_create(&philo[i], NULL, philosopher, &id_arr[i]);
        usleep(100000);
    }

    // jalankan selama 60 detik
    sleep(60);

    // hentikan threads: kita cancel them
    for (int i = 0; i < N; i++) pthread_cancel(philo[i]);
    for (int i = 0; i < N; i++) pthread_join(philo[i], NULL);

    printf("\n===== STATISTIK (60 detik) =====\n");
    int total = 0;
    for (int i = 0; i < N; i++) {
        printf("Filsuf-%d: Makan %d kali\n", i+1, eat_count[i]);
        total += eat_count[i];
    }
    if (total > 0)
        printf("✅ tidak ada deadlock!\n");
    else
        printf("⚠️ kemungkinan deadlock atau starvation!\n");

    // cleanup
    for (int i = 0; i < N; i++) pthread_mutex_destroy(&forks[i]);
    return 0;
}
```

```
vos@vos-QEMU-Virtual-Machine:~$ ./dining_limiter
=== DINING PHILOSOPHERS PROBLEM ===
Solusi: Resource Ordering

[00:00] 🧠 Filsuf--197263055 sedang berpikir...
[00:00] 🧠 Filsuf--197263051 sedang berpikir...
[00:00] 🧠 Filsuf--197263047 sedang berpikir...
[00:00] 🧠 Filsuf--197263043 sedang berpikir...
[00:00] 🍴 Filsuf--197263051 lapar, mencoba ambil sumpit...
[00:00] 🍴 Filsuf--197263051 mengambil sumpit kiri (-197263052)
Segmentation fault (core dumped)
```

Cara compile dan run :

`gcc -o dining_limiter dining_limiter.c`

`./dining_limiter`

4. Tabel Perbandingan Efisiensi dan Fairness

Pendekatan / Solusi	Efisiensi (Penggunaan Fork & CPU)	Fairness (Keadilan)	Dead lock	Starvation
Tanpa Pencegahan (Deadlock)	Rendah — seluruh thread menunggu tanpa progres	Tidak adil	Ya	Ya
Limiting Philosophers (maks. 4 dari 5 makan)	Tinggi — mencegah semua menunggu secara bersamaan	Cukup adil	Tidak	Hampir tidak ada
Even–Odd Rule (aturan ganjil–genap)	Sedang — tergantung urutan dan waktu akses	Cukup adil	Tidak	Bisa terjadi pada kondisi tertentu
Waiter / Semaphore	Sangat efisien — terkontrol	Paling adil	Tidak	Tidak

Analisis Mendalam

5.1 Penyebab Deadlock

Deadlock terjadi karena setiap filsuf memegang satu sumpit dan menunggu sumpit lain yang sedang digunakan oleh tetangganya.

Keempat syarat Coffman terpenuhi:

1. **Mutual Exclusion** – setiap sumpit hanya dapat digunakan oleh satu filsuf pada satu waktu.
2. **Hold and Wait** – setiap filsuf memegang satu sumpit sambil menunggu sumpit berikutnya.
3. **No Preemption** – sumpit tidak dapat diambil paksa dari filsuf lain.
4. **Circular Wait** – setiap filsuf menunggu sumpit yang dipegang oleh tetangga dalam lingkaran.

Contohnya, bila semua filsuf mengambil sumpit kiri bersamaan, tidak ada yang dapat melanjutkan ke sumpit kanan — sehingga sistem terhenti (deadlock).

5.2 Cara Kerja Setiap Solusi

- **Tanpa Solusi (Demonstrasi Deadlock):**
Semua filsuf mengambil sumpit kiri terlebih dahulu, lalu menunggu sumpit kanan. Akibatnya seluruh proses berhenti karena tidak ada yang dapat melanjutkan.
- **Solusi 1 – Limiting Philosophers:**
Hanya empat filsuf yang diizinkan mencoba makan secara bersamaan. Dengan begitu, satu filsuf selalu menunggu dan siklus penantian tidak pernah lengkap, sehingga deadlock berhasil dihindari.
- **Solusi 2 – Even–Odd Rule (Aturan Ganjil–Genap):**
Filsuf ganjil mengambil sumpit kiri terlebih dahulu, sedangkan filsuf genap mengambil sumpit kanan lebih dulu. Pola ini memutus rantai tunggu melingkar, sehingga deadlock jarang terjadi.
- **Solusi 3 – Waiter (Semaphore Global):**
Terdapat satu pengontrol (waiter) yang memberi izin makan hanya jika dua sumpit tersedia.

Dengan pengaturan ini, semua filsuf mendapat giliran secara bergantian dan deadlock sepenuhnya dapat dicegah.

5.3 Solusi yang Paling Adil

Pendekatan **Waiter / Semaphore Global** dinilai paling adil karena setiap filsuf mendapatkan giliran makan secara bergantian berdasarkan ketersediaan sumber daya.

Sementara itu, **Limiting Philosophers** masih cukup adil, meski beberapa filsuf mungkin menunggu lebih lama.

Even–Odd Rule lebih sederhana namun belum sepenuhnya merata dalam pembagian giliran.

5.4 Kemungkinan Terjadinya Starvation

- **Tanpa Solusi:** sangat mungkin — proses dapat menunggu tanpa batas waktu.
- **Limiting Philosophers:** kemungkinan kecil, namun tetap bisa terjadi jika penjadwalan tidak merata.
- **Even–Odd Rule:** masih berpotensi starvation karena distribusi giliran tidak selalu seimbang.
- **Waiter / Semaphore Global:** hampir tidak ada starvation karena pengaturan dilakukan secara adil dan bergiliran.

Kesimpulan dan Saran

6.1 Kesimpulan

Deadlock dapat terjadi ketika semua kondisi Coffman terpenuhi dalam sistem sinkronisasi.

Dalam kasus *Dining Philosophers Problem*, deadlock dapat dicegah melalui:

- Pengaturan urutan pengambilan sumber daya (*resource ordering*).
- Pembatasan jumlah proses yang aktif (*limiting philosophers*).

Penggunaan **mutex** dan **semaphore** sangat penting untuk menjaga sinkronisasi antar-thread, mencegah kebuntuan, dan memastikan sistem berjalan stabil.

6.2 Saran

- Gunakan pendekatan **semaphore global** pada sistem dengan banyak thread agar distribusi akses lebih adil.
- Tambahkan **timeout** atau mekanisme *retry* agar proses yang menunggu terlalu lama dapat dibatalkan otomatis.
- Kembangkan sistem **monitoring** untuk mendeteksi potensi deadlock lebih cepat dan mempermudah analisis saat debugging.

DAFTAR PUSTAKA

1. Abraham Silberschatz, Peter B. Galvin, dan Greg Gagne. 2018. *Operating System Concepts* Edisi ke-10. Wiley.
2. Andrew S. Tanenbaum dan Herbert Bos. 2015. *Modern Operating Systems* Edisi ke-4. Pearson Education.
3. William Stallings. 2018. *Operating Systems: Internals and Design Principles* Edisi ke-9. Pearson.
4. Coffman, E. G., Elphick, M. J., dan Shoshani, A. 1971. *System Deadlocks*. ACM Computing Surveys, Vol. 3, No. 2, Hal. 67–78.
5. GeeksforGeeks. 2023. *Dining Philosophers Problem in Operating System*. [Online]. Tersedia di: <https://www.geeksforgeeks.org/dining-philosophers-problem-in-operating-system/>
6. Tutorialspoint. 2023. *Deadlock in Operating System*. [Online]. Tersedia di: <https://www.tutorialspoint.com/deadlock-in-operating-system>